

Atividade Somativa - Teste do Servidor JAVA PI IV

- RA: 24019083 NOME: João Pedro Duarte Giatti
- RA: 24008214 NOME: Jean Yuki Kimura
- RA: 24014979 NOME: Jhenifer Laís Barbosa
- RA: 24002529 NOME: Eduarda Luiza Pinheiro Nepomuceno
- RA: 24020763 NOME: Thiago Volponi Cosmo

1. Aplicando a abordagem **INTRACLASSES**, escolha e aplique a técnica de particionamento (por estados, por atributos ou por categorias) que for mais adequada para duas classes do servidor desenvolvido pela equipe. Use as DICAS abaixo para escolher a melhor técnica de particionamento:

- Se a classe tiver estado importante, use o particionamento por estado.
- Se a classe não tiver estado relevante, mas possuir métodos que fazem cálculos importantes, indica-se o particionamento por categoria (de métodos).
- Se a classe não tiver estado e nem métodos de cálculos relevantes (provavelmente deve servir apenas para carregar e manter dados de cada instância), indica-se o particionamento por atributo.

Então, prepare e documente todas as sequencias de métodos que deverão ser exercitadas em cada Classe, usando a Tabela abaixo.

Classe	Particionamento adotado	Sequências INTRACLASSES (1 sequência por linha)
1) Rota	Por Atributos - A classe é um modelo de dados; testamos construtores, getters/setters e métodos equals, hashCode e toString.	S1: new Rota(uid, name, description, startLocation, endLocation, distance, recordTime, pontos) → getId(), getName(), getDescription(), getStartLocation(), getEndLocation(), getDistance(), getRecordTime(), getPoints() (verifica preenchimento correto de todos os atributos pelo construtor completo). S2: new Rota(uid, name, description, startLocation, endLocation, distance, recordTime, null) → getPoints() (verifica que, com lista de pontos nula, a classe inicializa uma lista vazia não nula). S3: new Rota() → setId(), setName(), setDescription(), setStartLocation(), setEndLocation(), setDistance(), setRecordTime(), setPoints() → leitura com getId(), getName(), ... (verifica atualização de atributos via setters). S4: rota1 = new Rota(...); rota2 = new Rota(...); → rota1.equals(rota2), rota1.hashCode(), rota2.hashCode() (verifica igualdade e hashCode para objetos com mesmos valores).

		S5: rota1 = new Rota(...); rota2 = new Rota(... com algum campo diferente ...) → rota1.equals(rota2) (verifica que objetos com algum campo diferente não são iguais). S6: new Rota(...); → toString() (verifica que a representação textual contém informações principais da rota).
2) ValidadorDeRota	Particionamento por categorias de validação. A classe expõe o método estático isValid(Rota) e os testes são organizados em categorias: campos obrigatórios, distância, tempo, pontos e coerência físico-matemática.	C1: isValid(null) (rota nula deve ser inválida). C2: rota = criarRotaValida(); → isValid(rota) (rota totalmente válida deve retornar true). C3: rota = criarRotaValida(); rota.setUid(""); → isValid(rota) (UID vazio/branco torna a rota inválida). C4: rota = criarRotaValida(); rota.setName(""); → isValid(rota) (nome vazio torna a rota inválida). C5: rota = criarRotaValida(); rota.setDescription(""); → isValid(rota) (descrição vazia/blank torna a rota inválida). C6: rota = criarRotaValida(); rota.setStartLocation(""); → isValid(rota) (local inicial vazio torna a rota inválida). C7: rota = criarRotaValida(); rota.setEndLocation(null); → isValid(rota) (local final nulo torna a rota inválida). C8: rota = criarRotaValida(); rota.setDistance(0); → isValid(rota) (distância zero é inválida). C9: rota = criarRotaValida(); rota.setDistance(-10); → isValid(rota) (distância negativa é inválida). C10: rota = criarRotaValida(); rota.setRecordTime(null); → isValid(rota) (tempo nulo é inválido). C11: rota = criarRotaValida(); rota.setRecordTime("abc"); → isValid(rota) (tempo em texto livre é inválido). C12: rota = criarRotaValida(); rota.setRecordTime("25:00"); → isValid(rota) (formato incorreto, sem segundos, é inválido). C13: rota = criarRotaValida();

		rota.setRecordTime("99:99:99"); → isValid(rota) (valores de hora/min/seg fora da faixa são inválidos). C14: rota = criarRotaValida(); rota.setDistance(20000); rota.setRecordTime("00:15:00"); → isValid(rota) (tempo fisicamente impossível para a distância torna a rota inválida). C15: rota = criarRotaValida(); rota.setPoints(null); → isValid(rota) (lista de pontos nula é inválida). C16: rota = criarRotaValida(); rota.setPoints(new ArrayList<>()); → isValid(rota) (lista de pontos vazia é inválida). C17: rota = criarRotaValida(); listaComPontoNull; rota.setPoints(listaComPontoNull); → isValid(rota) (lista contendo ponto null é inválida). C18: rota = criarRotaValida(); alterar um ponto para latitude = 100.0 → isValid(rota) (latitude fora de [-90, 90] torna a rota inválida). C19: rota = criarRotaValida(); alterar um ponto para longitude = -200.0 → isValid(rota) (longitude fora de [-180, 180] torna a rota inválida).
--	--	---

2. Usando as sequências preparadas no item anterior, automatize os testes unitários das classes (INTRA-CLASSES) no JUNIT 5, preparando os casos e dados de teste (entradas e saídas + asserções) necessários para testar cada sequência de métodos identificadas no item 2, para cada uma das cinco classes. Cole a seguir o código dos testes **INTRACLASSES**:

R:
Rota Testes

```
private Rota criarRotaValidaMock() { 18 usages  Jhenifer Lais Barbosa *
    List<LatLng> pontos = Arrays.asList(
        new LatLng(-23.561684, -46.625378),
        new LatLng(-23.562000, -46.626000),
        new LatLng(-23.563000, -46.627000)
    );

    return new Rota(
        "user123",
        "Corrida da manhã",
        "Treino Leve",
        "Parque A",
        "Parque B",
        5000.0,
        "00:25:00",
        pontos
    );
}
```

Figura 1 método para facilitar os testes, pois instancia um objeto valido e só alteramos para cada teste

```

@Test @Jhenifer Laís Barbosa
void construtorCompleto_QuandoParametrosValidos_PreencheTodosOsCamposCorretamente() {
    // Arrange
    String uid = "user123";
    String name = "Corrida da manhã";
    String description = "Treino leve";
    String startLocation = "Parque A";
    String endLocation = "Parque B";
    double distance = 5000.0;
    String recordTime = "00:25:00";
    List<LatLng> pontos = Arrays.asList(
        new LatLng(-23.5, -46.6),
        new LatLng(-23.6, -46.7)
    );

    // Act
    Rota rota = new Rota(uid, name, description, startLocation, endLocation, distance, recordTime, pontos);

    // Assert
    assertEquals(
        () -> assertEquals(uid, rota.getUid()),
        () -> assertEquals(name, rota.getName()),
        () -> assertEquals(description, rota.getDescription()),
        () -> assertEquals(startLocation, rota.getStartLocation()),
        () -> assertEquals(endLocation, rota.getEndLocation()),
        () -> assertEquals(distance, rota.getDistance()),
        () -> assertEquals(recordTime, rota.getRecordTime()),
        () -> assertEquals(pontos, rota.getPoints())
    );
}

@Test @Jhenifer Laís Barbosa
void construtorCompleto_QuandoListaDePontosForNull_InicializaListaVazia() {
    // Arrange
    String uid = "user123";
    String name = "Corrida";
    String description = "Desc";
    String startLocation = "A";
    String endLocation = "B";
    double distance = 1000.0;
    String recordTime = "00:05:00";

    // Act
    Rota rota = new Rota(uid, name, description, startLocation, endLocation, distance, recordTime, null);

    // Assert
    assertNotNull(rota.getPoints(), "A lista de pontos não deve ser nula");
    assertTrue(rota.getPoints().isEmpty(), "A lista de pontos deve ser vazia quando null for informado");
}

```

```

@Test @Jhenifer Laís Barbosa
void toString_QuandoChamado_ContemInformacoesPrincipais() {
    // Arrange
    Rota rota = criarRotaValidaMock();

    // Act
    String str = rota.toString();

    // Assert
    assertEquals(
        () -> assertTrue(str.contains("user123"), "toString deve conter o uid"),
        () -> assertTrue(str.contains("Corrida da manhã"), "toString deve conter o nome"),
        () -> assertTrue(str.contains("Parque A"), "toString deve conter a origem"),
        () -> assertTrue(str.contains("Parque B"), "toString deve conter o destino")
    );
}

```

```

@Test @Jhenifer Laís Barbosa
void setters_QuandoChamados_AtualizamCamposCorretamente() {
    // Arrange
    Rota rota = new Rota("", "", "", "", "", 0.0, "", null);

    String uid = "u1";
    String name = "Nome";
    String description = "Desc";
    String startLocation = "Origem";
    String endLocation = "Destino";
    double distance = 1234.5;
    String recordTime = "01:02:03";
    List<LatLng> pontos = new ArrayList<>();
    pontos.add(new LatLng(-23.0, -46.0));

    // Act
    rota.setUid(uid);
    rota.setName(name);
    rota.setDescription(description);
    rota.setStartLocation(startLocation);
    rota.setEndLocation(endLocation);
    rota.setDistance(distance);
    rota.setRecordTime(recordTime);
    rota.setPoints(pontos);

    // Assert
    assertEquals(
        () -> assertEquals(uid, rota.getUid()),
        () -> assertEquals(name, rota.getName()),
        () -> assertEquals(description, rota.getDescription()),
        () -> assertEquals(startLocation, rota.getStartLocation()),
        () -> assertEquals(endLocation, rota.getEndLocation()),
        () -> assertEquals(distance, rota.getDistance()),
        () -> assertEquals(recordTime, rota.getRecordTime()),
        () -> assertEquals(pontos, rota.getPoints())
    );
}

@Test @Jhenifer Laís Barbosa
void equals_QuandoObjetosPossuemMesmosValores_RetornaTrue() {
    // Arrange
    Rota rota1 = criarRotaValidaMock();
    Rota rota2 = criarRotaValidaMock();

    // Act
    boolean iguais = rota1.equals(rota2);

    // Assert
    assertTrue(iguais, "Rotas com os mesmos dados devem ser consideradas iguais");
    assertEquals(rota1.hashCode(), rota2.hashCode(), "hashCode deve ser igual para objetos iguais");
}

@Test @Jhenifer Laís Barbosa
void equals_QuandoAlgumCampoForDiferente_RetornaFalse() {
    // Arrange
    Rota rota1 = criarRotaValidaMock();
    Rota rota2 = criarRotaValidaMock();

    rota1.setUid("uidDiferente");

    // Act
    boolean iguais = rota1.equals(rota2);

    // Assert
    assertFalse(iguais, "Rotas com dados diferentes não devem ser consideradas iguais");
}

```

Validador de rotas testes

```
private Rota criarRotaValidaMock() { 18 usages  Jennifer Lais Barbosa *
    List<LatLng> pontos = Arrays.asList(
        new LatLng(-23.561684, -46.625378),
        new LatLng(-23.562000, -46.626000),
        new LatLng(-23.563000, -46.627000)
    );

    return new Rota(
        "user123",
        "Corrida da manhã",
        "Treino leve",
        "Parque A",
        "Parque B",
        5000.0,
        "00:25:00",
        pontos
    );
}
```

Figura 2 método para facilitar os testes, pois instancia um objeto e só alteramos para cada teste

```
@Test  Jennifer Lais Barbosa
void isValid_QuandoRotaForNull_RetornaFalse() {
    // Arrange
    Rota rota = null;

    // Act
    boolean valido = ValidadorDeRota.isValid(rota);

    // Assert
    assertFalse(valido);
}

@Test  Jennifer Lais Barbosa *
void isValid_QuandoRotaForValida_RetornaTrue() {
    // Arrange
    Rota rota = criarRotaValidaMock();

    // Act
    boolean valido = ValidadorDeRota.isValid(rota);

    // Assert
    assertTrue(valido);
}

@Test  Jennifer Lais Barbosa *
void isValid_QuandoUidForVazio_RetornaFalse() {
    // Arrange
    Rota rota = criarRotaValidaMock();
    rota.setUid("");

    // Act
    boolean valido = ValidadorDeRota.isValid(rota);

    // Assert
    assertFalse(valido);
}
```

```
@Test  Jennifer Lais Barbosa *
void isValid_QuandoNomeForVazio_RetornaFalse() {
    // Arrange
    Rota rota = criarRotaValidaMock();
    rota.setName("");

    // Act
    boolean valido = ValidadorDeRota.isValid(rota);

    // Assert
    assertFalse(valido);
}

@Test  Jennifer Lais Barbosa *
void isValid_QuandoDescricaoForVazia_RetornaFalse() {
    // Arrange
    Rota rota = criarRotaValidaMock();
    rota.setDescription("");

    // Act
    boolean valido = ValidadorDeRota.isValid(rota);

    // Assert
    assertFalse(valido);
}

@Test  Jennifer Lais Barbosa *
void isValid_QuandoLocalInicialForVazio_RetornaFalse() {
    // Arrange
    Rota rota = criarRotaValidaMock();
    rota.setStartLocation("");

    // Act
    boolean valido = ValidadorDeRota.isValid(rota);

    // Assert
    assertFalse(valido);
}

@Test  Jennifer Lais Barbosa *
void isValid_QuandoLocalFinalForVazio_RetornaFalse() {
    // Arrange
    Rota rota = criarRotaValidaMock();
    rota.setEndLocation(null);

    // Act
    boolean valido = ValidadorDeRota.isValid(rota);

    // Assert
    assertFalse(valido);
}

@Test  Jennifer Lais Barbosa *
void isValid_QuandoDistanciaForZero_RetornaFalse() {
    // Arrange
    Rota rota = criarRotaValidaMock();
    rota.setDistance(0.0);

    // Act
    boolean valido = ValidadorDeRota.isValid(rota);

    // Assert
    assertFalse(valido);
}

@Test  Jennifer Lais Barbosa *
void isValid_QuandoDistanciaForNegativa_RetornaFalse() {
    // Arrange
    Rota rota = criarRotaValidaMock();
    rota.setDistance(-10.0);

    // Act
    boolean valido = ValidadorDeRota.isValid(rota);

    // Assert
    assertFalse(valido);
}
```

```

@Test @JheniferLaisBarbosa *
void isValid_QuandoTempoForNulo_RetornaFalse() {
    // Arrange
    Rota rota = criarRotaValidaMock();
    rota.setRecordTime(null);

    // Act
    boolean valido = ValidadorDeRota.isValid(rota);

    // Assert
    assertFalse(valido);
}

@Test @JheniferLaisBarbosa *
void isValid_QuandoTempoForTextoLivre_RetornaFalse() {
    // Arrange
    Rota rota = criarRotaValidaMock();
    rota.setRecordTime("abc");

    // Act
    boolean valido = ValidadorDeRota.isValid(rota);

    // Assert
    assertFalse(valido);
}

@Test @JheniferLaisBarbosa *
void isValid_QuandoTempoEstiverEmFormatoIncorreto_RetornaFalse() {
    // Arrange
    Rota rota = criarRotaValidaMock();
    rota.setRecordTime("25:00"); // faltando parte dos segundos

    // Act
    boolean valido = ValidadorDeRota.isValid(rota);

    // Assert
    assertFalse(valido);
}

@Test @JheniferLaisBarbosa *
void isValid_QuandoTempoTiverNumerosForaDaFaixa_RetornaFalse() {
    // Arrange
    Rota rota = criarRotaValidaMock();
    rota.setRecordTime("99:99:99");

    // Act
    boolean valido = ValidadorDeRota.isValid(rota);

    // Assert
    assertFalse(valido);
}

@Test @JheniferLaisBarbosa *
void isValid_QuandoTempoForImpossivelParaDistancia_RetornaFalse() {
    // Arrange
    Rota rota = criarRotaValidaMock();
    rota.setDistance(20000.0); // 20 km
    rota.setRecordTime("00:15:00"); // 15 min -> impossible

    // Act
    boolean valido = ValidadorDeRota.isValid(rota);

    // Assert
    assertFalse(valido);
}

@Test @JheniferLaisBarbosa *
void isValid_QuandoListaDePontosForNula_RetornaFalse() {
    // Arrange
    Rota rota = criarRotaValidaMock();
    rota.setPoints(null);

    // Act
    boolean valido = ValidadorDeRota.isValid(rota);

    // Assert
    assertFalse(valido);
}

```

```

@Test @JheniferLaisBarbosa *
void isValid_QuandoListaDePontosForVazia_RetornaFalse() {
    // Arrange
    Rota rota = criarRotaValidaMock();
    rota.setPoints(new ArrayList<>());

    // Act
    boolean valido = ValidadorDeRota.isValid(rota);

    // Assert
    assertFalse(valido);
}

@Test @JheniferLaisBarbosa *
void isValid_QuandoAlgumPontoForNulo_RetornaFalse() {
    // Arrange
    List<LatLng> pontos = new ArrayList<>();
    pontos.add(new LatLng(-23.5, -46.6));
    pontos.add(null);

    Rota rota = criarRotaValidaMock();
    rota.setPoints(pontos);

    // Act
    boolean valido = ValidadorDeRota.isValid(rota);

    // Assert
    assertFalse(valido);
}

@Test @JheniferLaisBarbosa *
void isValid_QuandoLatitudeForInvalida_RetornaFalse() {
    // Arrange
    List<LatLng> pontos = Arrays.asList(
        new LatLng(100.0, -46.6), // latitude inválida
        new LatLng(-23.5, -46.7)
    );

    Rota rota = criarRotaValidaMock();
    rota.setPoints(pontos);

    // Act
    boolean valido = ValidadorDeRota.isValid(rota);

    // Assert
    assertFalse(valido);
}

@Test @JheniferLaisBarbosa *
void isValid_QuandoLongitudeForInvalida_RetornaFalse() {
    // Arrange
    List<LatLng> pontos = Arrays.asList(
        new LatLng(-23.5, -200.0), // longitude inválida
        new LatLng(-23.6, -46.7)
    );

    Rota rota = criarRotaValidaMock();
    rota.setPoints(pontos);

    // Act
    boolean valido = ValidadorDeRota.isValid(rota);

    // Assert
    assertFalse(valido);
}

```

3. Execute todos os testes **INTRACLASSES** automatizados no item 2. Anote e documente na tabela abaixo as falhas localizadas durante o teste, para as classes testadas:

Falha	Descrição	Possível Causa
Nenhuma falha encontrada	Todos os testes INTRACLASSES executados para as classes Rota e ValidadorDeRota passaram com sucesso.	Não se aplica. Implementação atual atende às regras de validação exercitadas pelos testes.

4. Usando a abordagem **INTERCLASSES**, testar o servidor desenvolvido, automatizando os testes necessários para percorrer o principal serviço provido pelo servidor desde a recepção da chamada, instanciação e chamada de métodos das classes participantes e a devolução do resultado à aplicação principal.
Além do casos de teste para o cenário NORMAL (funcionamento padrão), desenvolver pelo menos mais dois casos de teste para cenários de VARIAÇÃO do normal (retorno de erros, resultados alternativos, etc.).

Então, prepare e documente a sequencias de métodos que deverão ser exercitados para o NOMRAL e as duas variações

Cenário	Sequências INTERCLASSES para testar Cenário Solicitado
NORMAL	Objetivo: testar o fluxo normal em que uma rota válida é enviada do cliente para o servidor e o servidor responde que a rota é válida. Sequência (servidor): - usuarios = new ArrayList<Parceiro>(); (lista de clientes conectados). - aceitadoraDeConexao = new AceitadoraDeConexao(String.valueOf(PORT), usuarios); - aceitadoraDeConexao.start(); (thread do servidor começa a aceitar conexões na porta 3000). Sequência (cliente/aplicação): - Criar uma Rota rotaValida = new Rota(uid, name, description, startLocation, endLocation, distance=5000.0, recordTime="00:25:00", listaDePontosValidos); (todos os campos coerentes e pontos com latitude/longitude válidas). - Chamar o helper enviarRotaParaServidor(rotaValida). - Dentro do helper: Socket socket = new Socket(HOST, PORT); (cliente conecta no servidor). - Criar streams: DataOutputStream out = new DataOutputStream(socket.getOutputStream()); e DataInputStream in = new DataInputStream(socket.getInputStream()); - Enviar dados da rota para o servidor, na ordem: out.writeUTF(rota.getUid());, out.writeUTF(rota.getName());, out.writeUTF(rota.getDescription());, out.writeUTF(rota.getStartLocation());, out.writeUTF(rota.getEndLocation());, out.writeUTF(Double.toString(rota.getDistance()));, out.writeUTF(rota.getRecordTime());

	9. Enviar lista de pontos: <code>out.writeInt(pontos.size());</code> e, em loop, para cada <code>LatLng</code> ponto: <code>out.writeDouble(ponto.getLatitude());</code>, <code>out.writeDouble(ponto.getLongitude());</code> 10. <code>out.flush();</code> (envio efetivo). 11. Receber o resultado da validação remota: <code>String resposta = in.readUTF();</code> 12. No teste: verificar que <code>resposta.equals("true")</code> (servidor confirmou que a rota é válida).
VARIAÇÃO 1	Cenário: tempo impossível para a distância (rota fisicamente impossível). Espera-se que o servidor responda "false" por se tratar de um cenário de "cheating". Sequência (servidor): 1. Servidor já iniciado como no cenário NORMAL (<code>AceitadoraDeConexao.start()</code>). Sequência (cliente/aplicação): 2. Criar Rota <code>rotaImpossivel = new Rota(uid, "Rota Impossível", "20km em 15min", "Ponto A", "Ponto B", distance=20000.0, recordTime="00:15:00", listaDePontosValidos);</code> (distância muito grande para um tempo muito curto). 3. Chamar <code>enviarRotaParaServidor(rotaImpossivel);</code> 4. Dentro do helper, repetir a mesma sequência de envio do cenário NORMAL (criar Socket, criar <code>DataOutputStream/DataInputStream</code>, escrever todos os campos com <code>writeUTF</code>, enviar quantidade de pontos com <code>writeln</code>, enviar cada latitude/longitude com <code>writeDouble</code>, <code>flush</code>). 5. Ler resposta do servidor: <code>String resposta = in.readUTF();</code> 6. No teste: verificar que <code>resposta.equals("false")</code> (servidor detecta que a combinação distância x tempo não é plausível e se enquadra como cheat).
VARIAÇÃO 2	Cenário: pontos geográficos inválidos (coordenadas fora da faixa). Espera-se que o servidor responda "false" por se tratar de um cenário dados inválidos. Sequência (servidor): 1. Servidor já iniciado como no cenário NORMAL (<code>AceitadoraDeConexao</code> em execução na porta 3000). Sequência (cliente/aplicação): 2. Montar lista de pontos com pelo menos um ponto inválido, por exemplo: <code>List<LatLng> pontosInvalidos = Arrays.asList(new LatLng(200.0, -46.625378), new LatLng(-23.562000, -46.626000));</code> (latitude 200.0 é inválida). 3. Criar Rota <code>rotaComPontosInvalidos = new Rota(uid, "Rota Com Pontos Inválidos", "Teste de pontos inválidos", "Origem X", "Destino Y", distance=5000.0, recordTime="00:20:00", pontosInvalidos);</code> 4. Chamar <code>enviarRotaParaServidor(rotaComPontosInvalidos);</code>

	5. Dentro do helper, repetir sequência: new Socket(HOST, PORT), criação de DataOutputStream e DataInputStream, envio dos campos com writeUTF, envio do tamanho da lista e de cada LatLng com writeInt + writeDouble, flush. 6. Ler resposta do servidor: String resposta = in.readUTF(); 7. No teste: verificar que resposta.equals("false") (servidor detecta pontos da rota inválidos).
--	--

5. Automatize o teste, preparando o caso e os dados de teste (entradas e saídas + asserções) necessários para testar as 3 sequências de métodos documentada. Cole a seguir o código dos testes **INTERCLASSES**:

R:

```
private static final String HOST = "localhost"; 1 usage
private static final int PORT = 3000; 2 usages

private static AceitadoraDeConexao aceitadoraDeConexao; 5 usages
private static ArrayList<Parceiro> usuarios; 2 usages

@BeforeAll @Jhenifer Laís Barbosa
static void servidor_QuandoIniciado_FicaProntoParaReceberConexoes() throws Exception {
    // Arrange
    usuarios = new ArrayList<>();

    // Act
    aceitadoraDeConexao = new AceitadoraDeConexao(String.valueOf(PORT), usuarios);
    aceitadoraDeConexao.start();
    Thread.sleep(200); // pequeno delay para subir a porta

    // Assert (implícito): se der problema, os testes de conexão vão falhar
}

@AfterAll @Jhenifer Laís Barbosa
static void servidor_QuandoFinaliza_InterrompeThreadDeAceitacao() {
    // Arrange & Act
    if (aceitadoraDeConexao != null && aceitadoraDeConexao.isAlive()) {
        aceitadoraDeConexao.interrupt();
    }
    // Assert: encerramento acontece ao final dos testes
}
```

Figura 1 atributos e métodos para usar nos testes, para subir o servidor e fecha-lo após os testes

```
@Test @Jhenifer Laís Barbosa
void validacaoRemota_QuandoRotaValida_RetornaTrue() throws Exception {
    // Arrange
    Rota rotaValida = new Rota(
        "user123",
        "Corrida da Manhã",
        "Treino leve de 5k",
        "Parque A",
        "Parque B",
        5000.0,
        "00:25:00",
        Arrays.asList(
            new LatLng(-23.561684, -46.625378),
            new LatLng(-23.562000, -46.626000),
            new LatLng(-23.563000, -46.627000)
        )
    );

    // Act
    String resposta = enviarRotaParaServidor(rotaValida);

    // Assert
    assertEquals("true", resposta);
}
```

```

@Test & Jhenifer Laís Barbosa
void validacaoRemota_QuandoTempoImpossivelParaDistancia_RetornaFalse() throws Exception {
    // Arrange
    Rota rotaImpossivel = new Rota(
        "user123",
        "Rota Impossivel",
        "20km em 15min",
        "Ponto A",
        "Ponto B",
        20000.0,
        "00:15:00", // impossivel
        Arrays.asList(
            new LatLng(-23.561684, -46.625378),
            new LatLng(-23.562000, -46.626000)
        )
    );

    // Act
    String resposta = enviarRotaParaServidor(rotaImpossivel);

    // Assert
    assertEquals("false", resposta);
}

@Test & Jhenifer Laís Barbosa
void validacaoRemota_QuandoPontosInvalidos_RetornaFalse() throws Exception {
    // Arrange
    List<LatLng> pontosInvalidos = Arrays.asList(
        new LatLng(200.0, -46.625378), // latitude inválida
        new LatLng(-23.562000, -46.626000)
    );

    Rota rotaComPontosInvalidos = new Rota(
        "user123",
        "Rota com pontos inválidos",
        "Teste de pontos fora do intervalo",
        "Lugar X",
        "Lugar Y",
        3000.0,
        "00:20:00",
        pontosInvalidos
    );

    // Act
    String resposta = enviarRotaParaServidor(rotaComPontosInvalidos);

    // Assert
    assertEquals("false", resposta);
}

```

```

private String enviarRotaParaServidor(Rota rota) throws Exception {
    try (Socket socket = new Socket(HOST, PORT);
        DataOutputStream out = new DataOutputStream(socket.getOutputStream());
        DataInputStream in = new DataInputStream(socket.getInputStream())) {

        // Envio dos campos básicos
        out.writeUTF(rota.getUid());
        out.writeUTF(rota.getName());
        out.writeUTF(rota.getDescription());
        out.writeUTF(rota.getStartLocation());
        out.writeUTF(rota.getEndLocation());
        out.writeUTF(Double.toString(rota.getDistance()));
        out.writeUTF(rota.getRecordTime());

        // Envio da lista de pontos
        List<LatLng> pontos = rota.getPoints();
        if (pontos == null) {
            pontos = new ArrayList<>();
        }

        out.writeInt(pontos.size());
        for (LatLng ponto : pontos) {
            out.writeDouble(ponto.getLatitude());
            out.writeDouble(ponto.getLongitude());
        }

        out.flush();

        // Resposta do servidor
        return in.readUTF();
    }
}

```

Figura 3 método helper para enviar rota para o servidor

6. Execute todos os testes **INTERCLASSES** automatizados no item 5. Anote e documente na tabela abaixo as falhas localizadas durante o teste, para as classes testadas:

Falha	Descrição	Possível Causa
Nenhuma falha encontrada	Todos os testes INTERCLASSES executados para o cenário NORMAL e variações passaram com sucesso. O servidor respondeu corretamente "true" para rota válida e "false" para rotas inválidas(dados invalidos ou cheating).	Não se aplica. A implementação atual do servidor está consistente com as regras de validação exercitadas pelos testes.